

附件 2:

第十九届全国大学生信息安全竞赛（作品赛） 暨第三届“长城杯”网数智安全大赛（作品赛） 作品报告

☒ 命题赛道

☐ 自由赛道

作品名称: 链甲（AgentArmor）：面向 LLM-Agent 长期记忆污染的全链防御系统

电子邮箱: _____

提交日期: _____

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用 A4 纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5 倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。

目 录

摘要	1
第一章 作品概述	2
第二章 作品设计与实现.....	3
2.1 系统整体设计	3
2.2 可信记忆对象与安全边界.....	4
2.3 类型一：框架自动写入防御模块	4
2.3.1 写入期防护：从候选文本到可信记忆条目	4
2.3.2 检索期防护：从历史命中到上下文准入	7
2.3.3 类型一端到端流程	9
2.4 类型二：Agent 自主写入防御模块	10
2.4.1 写入决策前审查	10
2.4.2 写入后可信性与检索期放大控制	12
2.4.3 类型二端到端流程	14
2.5 类型三：反思/合成写入防御模块.....	16
2.5.1 反思写入前的意图筛查	16
2.5.2 反思记忆的检索期复核	17
2.6 类型四：外部/开发者批量写入防御模块	18
2.6.1 外部文档进入知识库前的连续门控	19
2.7 类型五：Multi-Agent 共享写入防御模块	20
2.7.1 共享记忆污染链路建模	20
2.7.2 共享写入网关与候选事实筛查	21
2.7.3 检索审计、执行仲裁与撤销机制	23

第三章	作品测试与分析	26
3.1	测试目标	26
3.2	实验环境与数据设置	27
3.3	端到端攻击链验证	31
3.4	场景化评测结果	31
3.4.1	类型四：供应链与知识库污染	32
3.4.2	类型五：多 Agent 共享记忆	33
3.5	消融与误伤分析	34
3.6	运行开销	35
3.7	综合结论与扩展方向	36
第四章	创新性说明	38
4.1	问题定义层面的创新：从“输入安全”到“记忆安全”的范式转变	38
4.2	架构层面的创新：独立记忆安全网关（Memory Firewall）	38
4.3	检测机制的创新：同步规则过滤与异步语义免疫的分层协同防御	39
4.3.1	同步规则过滤层	39
4.3.2	异步语义免疫层	39
4.4	主动免疫机制的创新：基于 LLM 的沙箱模拟推理验证	39
4.5	数据结构的创新：可信记忆对象（Trusted MemoryEntry）	40
4.6	密码学应用的创新：Ed25519 来源签名保障记忆完整性	40
4.7	审计机制的创新：哈希链接的追加式审计链	41
4.8	防护时效的创新：周期扫描实现持续安全而非一次性过滤	41
4.9	隐私保护策略的创新：语义保留的 PII 动态脱敏	42
4.10	整体创新性总结	42
第五章	总结	43
	参考文献	44

摘要

(请简要说明创作本作品之动机、功能、特性、创新处、实用性)

第一章 作品概述

(建议包括：背景分析、相关工作、特色描述及应用前景分析等)

第二章 作品设计与实现

2.1 系统整体设计

链甲（AgentArmor）面向具备长期记忆能力的 LLM-Agent 系统，解决的核心问题不是单轮输入是否存在提示注入，而是恶意内容是否会沿着“写入、固化、检索、执行”的记忆生命周期被长期保存，并在未来任务中被正常检索后触发危险行为。传统提示注入防护主要作用于当前输入，而 AgentArmor 将防护边界前移至记忆写入入口，并后移至检索和工具执行阶段，从而构建覆盖长期状态的安全防线。

本章按照“总体架构—安全边界—分类型防护—关键算法”的逻辑展开。首先给出 AgentArmor 在长期记忆生命周期中的部署位置，然后说明可信记忆对象与写入、检索、执行三道安全边界，最后依次介绍五类风险场景下的防护模块与实现流程。第三章与本章保持对应关系，围绕五类场景分别给出测试目标、测试环境、测试方案、消融实验、基线对比和结果分析，从而体现作品作为完整工程系统的可实现性与可验证性。

从工程位置看，AgentArmor 不是替换底座大模型，也不是将检测逻辑硬编码进某一个 Agent 框架，而是在 Agent 框架、反思器、外部知识库导入管线、多 Agent 共享记忆和高影响工具执行器之间插入记忆安全网关。所有候选记忆进入长期存储前都需要经过来源标注、风险检测、写入门控和审计记录；所有被检索出的记忆重新进入上下文前，还需要经过完整性验证、可信重排、语义复核和工具执行仲裁。

系统总体流程可抽象为：

$$X \xrightarrow{\text{Ingress}} C \xrightarrow{\text{Write Guard}} M \xrightarrow{\text{Retrieval Guard}} R \xrightarrow{\text{Execution Guard}} A, \quad (2.1)$$

其中， X 表示用户输入、外部网页、工具返回、批量文档或其他 Agent 产出的原始内容； C 表示候选记忆； M 表示长期记忆库； R 表示被检索并准备进入上下文的记忆集合； A 表示最终回答或工具动作。系统的安全目标是保证每一阶段的状态转换都满足来源可追溯、内容可验证、权限可约束和异常可审计。

2.2 可信记忆对象与安全边界

AgentArmor 将长期记忆条目抽象为可信记忆对象。每条记忆不再只是向量数据库中的一段文本，而是由内容、来源、信任分、安全状态和审计记录共同构成。其核心字段包括：内容哈希、来源标识、来源类型、会话哈希、时间戳、信任分、签名值、隔离状态、隔离原因以及追加式审计事件。内容哈希用于检测存储后的离线篡改，来源字段用于区分用户输入、工具返回、网页内容、反思摘要和系统生成内容，信任分用于参与写入门控和检索重排，审计链用于记录从创建、检测、写入、检索到隔离的完整生命周期。

系统将记忆生命周期拆分为三个关键安全边界。第一是写入边界，即候选内容从短期上下文进入长期记忆之前的过滤与门控；第二是检索边界，即长期记忆被召回并进入当前上下文之前的验签、重排和风险复核；第三是执行边界，即检索内容即将影响邮件、文件、数据库、代码仓库或命令执行等高影响工具之前的授权审查。三道边界相互独立，即使某一阶段出现漏检，后续阶段仍能提供二次防护。

2.3 类型一：框架自动写入防御模块

类型一面向 Agent 框架中的自动记忆管理机制。此类框架常通过 `save_context()`、`ConversationBufferMemory` 等组件把历史对话自动写入记忆库。风险内容发起方不需要诱导 Agent 调用显式写入工具，只需在普通对话中嵌入“以后每次都执行某动作”“请记住某隐藏规则”等隐性指令，就可能使风险内容被框架底层自动保存。因此，类型一模块的关键防御对象是框架自动写入前后的记忆流，而不是 Agent 的工具调用决策。

围绕该路径，系统设置三个风险节点：N-W1 位于框架调用 `save_context()` 之前，风险内容可能在候选写入内容中嵌入指令性句子或制造大量相似写入；N-W2 位于风险记录写入存储之后，存在离线篡改历史条目的可能；N-R1 位于未来用户查询触发检索时，高相似度后门记忆或异常聚簇可能获得过高排序权重。围绕这三个节点，类型一模块实现 D-A 至 D-F 六个防御组件，并将其串联为写入期和检索期两个闭环。

2.3.1 写入期防护：从候选文本到可信记忆条目

D-A 是写前 Token 净化模块，部署在框架自动保存之前。该模块不直接删除整段候选记忆，而是先把候选文本拆分为句子级单元，再识别其中的指令性片段。这样做

的原因是：用户对话中可能同时包含可保存事实和不可保存指令，若整体拒绝会降低记忆可用性，若整体写入又会放大持久化风险。因此，算法 1 采用“句子级识别—比例阈值阻断—剩余内容回写”的策略实现细粒度净化。

算法 1: 类型一 D-A 写前 Token 净化

输入： 候选写入内容 m ，净化策略 $strategy$ ，移除比例上限 ρ_{max}

输出： $v_A \in \{\text{PASS}, \text{FLAG}, \text{BLOCK}\}$ ，净化后内容 m'

注：该步骤位于框架自动保存之前，用于把候选内容中的指令片段从可写入事实中剥离。

```
1:  $S \leftarrow \text{SentenceSplit}(m)$  ▷ 先把候选记忆拆成句子级单元
2:  $S_{cmd} \leftarrow \{s \in S : D_A^{strategy}(s) = 1\}$  ▷ 定位不应被长期保存的指令性片段
3:  $ratio \leftarrow |S_{cmd}|/|S|$ 
4: if  $ratio > \rho_{max}$  then
5:   return Block
6: end if
7:  $m' \leftarrow \text{Join}(S \setminus S_{cmd})$ 
8: if  $|S_{cmd}| > 0$  then
9:   return Flag,  $m'$ 
10: end if
11: return Pass,  $m'$ 
```

经过 D-A 处理后，候选内容已经去除了明显指令片段，但这并不意味着它一定具有长期保存价值。D-B 选择性写入策略进一步评估候选内容是否具有新颖性、是否与已有记忆形成异常密集簇、是否超过会话写入频率上限。该模块主要解决自动记忆组件“凡对话皆保存”的问题，使记忆库只保留对未来任务有价值且风险可控的内容。

算法 2: 类型一 D-B 选择性写入策略

输入： 候选内容 m ，写入上下文 wc ，记忆库索引 $\mathcal{M}$

输出： $v_B \in \{\text{PASS}, \text{BLOCK}\}$

注：该步骤判断候选内容是否值得写入，避免自动记忆组件保存低价值或批量相似内容。

```
1: if  $session\_count > max\_session\_writes$  then
2:   return Block
```

```

3: end if
4:  $\mathbf{e} \leftarrow \text{Embed}(m)$ 
5:  $\text{density} \leftarrow \text{ClusterDensity}(\mathbf{e}, \mathcal{M}, r_{\text{cluster}})$   $\triangleright$  检查是否存在批量相似写入
6: if  $\text{density} > \rho_{\text{limit}}$  then
7:   return Block
8: end if
9:  $\text{novelty} \leftarrow 1 - \text{TopKMeanSim}(\mathbf{e}, \mathcal{M}, k)$ 
10:  $V \leftarrow \alpha \text{novelty} - \beta(1 - \text{novelty}) - \gamma \text{risk}(m)$ 
11: if  $V < \tau_{\text{write}}$  then
12:   return Block
13: end if
14: return Pass

```

当候选内容通过 D-A 和 D-B 后，系统才允许其进入长期记忆库。为了避免写入后的离线篡改，D-C 将内容哈希、来源标签、触发查询哈希、时间戳和前序哈希共同纳入 HMAC 计算，形成链式完整性证据。这样，后续检索阶段不仅能验证单条记忆是否被篡改，还能发现审计链是否被截断或重排。

算法 3: 类型一 D-C 存储完整性链

输入： 内容 content ，溯源标签 ℓ ，触发查询哈希 h_{tq} ，写入时间 t ，前序哈希 h_{prev} ，存储哈希 h_{stored}

输出： 写入端链式哈希 h_i ；检索端完整性判决

注：该步骤为已放行内容绑定哈希链，后续检索时可发现离线篡改。

```

1:  $\text{payload} \leftarrow \text{JSON}(id \parallel \text{SHA256}(\text{content}) \parallel \ell \parallel h_{tq} \parallel t \parallel h_{\text{prev}})$   $\triangleright$  把内容、来源和前序哈希共同纳入签名
2:  $h_i \leftarrow \text{HMAC}_K(\text{payload})$ 
3:  $\text{memory.insert}(\text{content}, h_i, h_{\text{prev}})$ 
4:  $h_{\text{computed}} \leftarrow \text{HMAC}_K(\text{JSON}(\dots, h_{\text{prev}}))$ 
5: if  $h_{\text{computed}} \neq h_{\text{stored}}$  then
6:    $\text{quarantine}(d)$ 
7:   return Block
8: end if

```

9: return Pass

上述三个流程构成类型一的写入期防线：D-A 负责“能不能写入内容本身”，D-B 负责“是否值得长期保存”，D-C 负责“写入后能否证明未被篡改”。三者依次执行，使框架自动写入从无条件保存变为受控保存。

2.3.2 检索期防护：从历史命中到上下文准入

类型一的检索期防护面向另一个核心问题：即便某条风险记忆已经进入长期库，也不应在未来查询中被直接高权重召回。D-D 时序衰减重排序首先根据写入时间和信任分降低陈旧条目的权重，避免早期写入的内容长期占据上下文入口。

算法 4: 类型一 D-D 时序衰减重排序

输入：检索条目列表 $\mathcal{E}$ ，当前时间 t_{now} ，衰减速率 λ ，下界 w_{min}

输出：按新权重降序排列的条目列表 $\mathcal{E}'$

注：该步骤在检索期降低陈旧或低信任记忆的排序权重。

```
1: for all  $e_i \in \mathcal{E}$  do
2:    $\Delta t \leftarrow t_{now} - e_i.t_{write}$ 
3:    $e_i.w \leftarrow e_i.w \cdot \max(\exp(-\lambda\Delta t), w_{min})$ 
4: end for
5:  $\mathcal{E}' \leftarrow \text{Sort}(\mathcal{E}, \text{key} = w, \text{descending})$ 
6: return  $\mathcal{E}'$ 
```

仅依靠时序衰减并不足以判断检索命中是否真正服务于当前任务。D-E 检索对齐核查进一步比较用户任务与检索条目的嵌入相似度，并对边界样本调用 LLM 进行语义复核。该模块的重点不是重新判断条目是否曾经安全，而是判断它在当前查询中是否仍然“任务相关、无隐藏指令、可作为证据进入上下文”。

算法 5: 类型一 D-E 检索对齐核查

输入：检索条目列表 $\mathcal{E}$ ，用户任务 Q ，任务嵌入 $\mathbf{e}_Q$

输出：核查后条目列表

注：该步骤检查检索命中是否服务于当前任务，而不是夹带隐藏指令。

```
1: for all  $e_i \in \mathcal{E}$  do
2:    $s_{emb} \leftarrow \cos(E(e_i), \mathbf{e}_Q)$ 
```

```

3:   if  $s_{emb} \geq \tau_{pass}$  then
4:       continue
5:   else if  $s_{emb} < \tau_{susp}$  then
6:        $e_i.w \leftarrow e_i.w \cdot \delta_{down}$ 
7:       continue
8:   end if
9:    $(aligned, conf, hidden) \leftarrow \text{LLM\_Judge}(e_i, Q)$ 
10:  if not aligned or hidden then
11:       $e_i.w \leftarrow e_i.w \cdot \delta_{down}$ 
12:       $e_i.flagged \leftarrow \text{True}$ 
13:  end if
14: end for
15: return  $\mathcal{E}$ 

```

除单条记忆外，AgentArmor 还对整体记忆分布进行健康检查。D-F 记忆分布异常检测通过 PCA 分布直方图、KL 散度和 DBSCAN 聚簇分析识别批量相似写入和异常聚簇。该模块适合发现单条内容看似无害、但整体上呈现集中投毒趋势的风险。

算法 6: 类型一 D-F 记忆分布异常检测

输入： 记忆库嵌入矩阵 $\mathbf{E}$ ，基线分布 P_{base} ，阈值 τ_{KL} 、 $\rho_{cluster}$

输出： 分布健康判决 $v_F \in \{\text{PASS}, \text{FLAG}, \text{BLOCK}\}$

注：该步骤从整体分布角度发现聚簇投毒或异常批量写入。

```

1:  $P_{cur} \leftarrow \text{PCAHistogram}(\mathbf{E}, n_{bins})$ 
2:  $kl \leftarrow \text{KL}(P_{cur} \| P_{base})$ 
3: if  $kl > \tau_{KL}$  then
4:     return  $action\_on\_anomaly$ 
5: end if
6:  $\mathcal{C} \leftarrow \text{DBSCAN}(\mathbf{E}, \epsilon, min\_samples)$ 
7: if  $\max_c |\mathcal{C}_c| / \text{mean}|\mathcal{C}| > \rho_{cluster}$  then
8:     return Flag
9: end if
10:  $P_{base} \leftarrow (1 - \epsilon_{ema})P_{base} + \epsilon_{ema}P_{cur}$ 

```

11: **return** Pass

2.3.3 类型一端到端流程

单点模块只有嵌入实际框架调用路径中才具有工程意义。算法 7 将 D-A、D-B、D-C 与通用检测器组合为自动写入路径，位于框架保存上下文之前；算法 8 将验签、重排、对齐核查和分布扫描组合为检索路径，位于历史记忆重新进入上下文之前。

算法 7: 类型一 AutoWrite 写入路径防御

输入： 候选条目 e ，写入上下文 wc ，记忆库 $\mathcal{M}$

输出： $verdict \in \{\text{PASS}, \text{BLOCK}\}$

注：该流程串联写入期节点，只有通过净化、价值判断和完整性绑定的内容才能持久化。

```
1:  $v_A \leftarrow D_A.\text{sanitize}(e)$ 
2: if  $v_A = \text{BLOCK}$  then
3:   return Block
4: end if
5:  $v_B \leftarrow D_B.\text{check}(e, wc, \mathcal{M})$ 
6: if  $v_B = \text{BLOCK}$  then
7:   return Block
8: end if
9:  $[D1 \rightarrow D2 \rightarrow D3 \rightarrow D4].\text{check}(e, wc)$ 
10:  $h_i \leftarrow D_C.\text{bind\_chain}(e)$ 
11:  $D_F.\text{update}(e)$ 
12: return Pass
```

写入路径完成后，记忆库中的条目已经具备内容净化、来源标签和完整性证据。检索路径则在每次命中历史记忆时执行二次复核，保证被召回的内容只作为与当前任务相关的上下文证据，而不是隐式指令。

算法 8: 类型一 AutoWrite 检索路径防御

输入： 检索条目列表 $\mathcal{E}$ ，用户任务 Q

输出： 处理后条目列表 $\mathcal{E}'$

注：该流程串联检索期节点，确保历史记忆进入上下文前再次接受安全复核。

```
1:  $\mathcal{E} \leftarrow D_C.verify(\mathcal{E})$ 
2:  $\mathcal{E} \leftarrow [D4.verify + D5.scan](\mathcal{E})$ 
3:  $\mathcal{E} \leftarrow D_D.rerank(\mathcal{E})$ 
4:  $\mathcal{E} \leftarrow D_E.verify(\mathcal{E}, Q)$ 
5:  $v_F \leftarrow D_F.scan()$ 
6: if  $v_F = \text{BLOCK}$  then
7:   return  $\emptyset$ 
8: end if
9: return  $\mathcal{E}$ 
```

2.4 类型二：Agent 自主写入防御模块

类型二对应 Agent-Initiated Write 场景，即外部输入通过正常查询接口诱导 Agent 自己调用长期记忆写入工具。与类型一不同，类型二不依赖框架后台自动保存，而是利用 Agent 的规划和工具调用能力，让风险内容以“Agent 自己认为应该保存”的形式进入长期记忆。原始材料中将该风险场景抽象为 MINJA 污染链路，覆盖明显写入指令、压缩残留标记、无 indication 的纯内容诱导以及群体 Hub 投毒四类变体。

类型二模块围绕三个风险节点展开。N1 是 Agent 决定调用写入工具之前的写入决策点；N2 是记录写入后到首次被检索前的存储可信性阶段；N3 是未来受害查询触发检索时的放大阶段。系统对应实现 D1 至 D6 六个组件：D1 写入意图检测，D2 反事实因果归因审查，D3 前瞻行为仿真验证，D4 信息流溯源签名，D5 向量空间 Hub 异常检测，D6 工具调用任务对齐检查。

2.4.1 写入决策前审查

类型二的第一道防线是判断 Agent 是否被异常诱导而准备写入记忆。D1 写入意图检测优先审查 indication prompt；如果不存在显式 indication，则回退到 triggering query。该设计保证了显式诱导和隐式诱导都能在工具调用之前被审查。

算法 9: 类型二 D1 写入意图检测

输入： q : 非空时为 indication prompt，否则为 triggering query

输出: $v_1 \in \{\text{PASS}, \text{BLOCK}\}$

注: 该步骤在 Agent 调用记忆工具前识别异常写入意图。

- 1: $d_1 \leftarrow \text{indication prompt}$ **if** indication prompt $\neq \emptyset$ **else** triggering query $\triangleright$ 优先审查显式写入诱导; 缺失时回退到触发查询
- 2: $v_1 \leftarrow D_1.\text{check}(d_1)$ $\triangleright$ 关键词、语义子空间和 LLM 级联检测
- 3: **if** $v_1 = \text{BLOCK}$ **then**
- 4: **return** Block
- 5: **end if**
- 6: **return** Pass

D1 能识别明显写入意图, 但不能证明候选记忆是否真的由诱导语句造成。D2 反事实因果归因审查通过删除 indication prompt 或 user goal 后重新评分, 估计候选记忆对诱导语句的依赖程度。如果候选内容在删除诱导语句后显著失去合理性, 则说明其主要来源不是用户真实目标, 而是异常写入诱导。

算法 10: 类型二 D2 反事实因果归因审查

输入: 写入上下文 wc , 含 S 、indication prompt、user goal、candidate content

输出: $v_2 \in \{\text{PASS}, \text{FLAG}, \text{BLOCK}\}$

注: 该步骤通过删除诱导语句的反事实比较, 判断候选记忆是否由诱导造成。

- 1: $p_{full} \leftarrow \text{ProxyLLM.score}(S, \text{candidate_content})$
- 2: $p_{ind} \leftarrow \text{ProxyLLM.score}(S \setminus \{\text{indication_prompt}\}, \text{candidate_content})$
- 3: $p_{user} \leftarrow \text{ProxyLLM.score}(S \setminus \{\text{user_goal}\}, \text{candidate_content})$
- 4: $DS \leftarrow (p_{full} - p_{ind}) - (p_{full} - p_{user})$ $\triangleright$ 衡量候选内容是否主要由诱导语句驱动
- 5: **if** $DS > \theta$ **then**
- 6: **return** Block
- 7: **end if**
- 8: **if** $|DS| < \delta$ **then**
- 9: **return** Flag
- 10: **end if**
- 11: **return** Pass

对于 D2 给出的边界样本，系统不立即写入真实记忆库，而是调用 D3 前瞻行为仿真。该模块把候选记忆放入隔离沙箱中，模拟未来查询命中该记忆后的 Agent 行为，并检查是否会引发外发邮件、泄露敏感信息、越权调用工具等风险后果。这样，系统能够从“未来使用效果”而非仅从“当前文本形式”判断候选内容是否安全。

算法 11: 类型二 D3 前瞻行为仿真验证

输入： 候选写入内容 m ，仿真上下文数量 n

输出： $v_3 \in \{\text{PASS}, \text{BLOCK}\}$

注：该步骤在沙箱中预演候选记忆未来被使用后的行为后果。

```
1:  $\mathcal{C} \leftarrow \text{GenerateContexts}(m, n)$ 
2:  $unsafe \leftarrow 0$ 
3: for all  $c_i \in \mathcal{C}$  do
4:    $result \leftarrow \text{LLM\_Judge}(m, c_i)$ 
5:   if  $result.is\_unsafe$  then
6:      $unsafe \leftarrow unsafe + 1$ 
7:   end if
8: end for
9: if  $unsafe > 0$  then
10:   return Block
11: end if
12: return Pass
```

2.4.2 写入后可信性与检索期放大控制

当候选内容经过前置审查后，D4 信息流溯源签名为其绑定来源标签、触发查询哈希和 HMAC 签名。该步骤将“谁产生了这条记忆、由什么上下文触发、是否被篡改”固化为可验证证据。

算法 12: 类型二 D4 信息流溯源签名

输入： 内容 $content$ ，安全标签 ℓ_r ，触发查询 tq ，检索记录 d

输出： 写入端带签名标签的记录；检索端异常记录隔离

注：该步骤为候选记忆绑定来源与完整性证据。

```
1:  $sig \leftarrow \text{HMAC}_k(content \parallel \ell_r \parallel \text{SHA256}(tq))$ 
```



```

2:  $tag \leftarrow (\ell_r, sig)$ 
3:  $memory.insert(content, tag)$ 
4:  $expected \leftarrow HMAC_k(d.content || d.label || d.query\_hash)$ 
5: if  $d.sig \neq expected$  then
6:    $quarantine(d)$ 
7: end if

```

D5 面向检索期的 Hub 异常问题。若某些条目在近期大量不同查询中都频繁进入 top-k，或形成异常密集的语义团簇，则该条目可能承担了跨任务放大的风险中转作用。D5 通过命中频次统计和图团检测识别此类异常条目，并把可疑记录交给隔离或降权流程。

算法 13: 类型二 D5 向量空间 Hub 异常检测

输入： 记忆库 M ，近期查询日志 Q_{recent} ，超参 $\alpha, \epsilon, k_{min}$

输出： 告警集合 S_{alert}

注：该步骤在向量空间中发现被异常频繁检索的 Hub 条目。

```

1: for all  $d \in M$  do
2:    $h[d] \leftarrow |\{q \in Q_{recent} : d \in top_k(q)\}|$ 
3: end for
4:  $\tau \leftarrow mean(h) + \alpha \cdot std(h)$ 
5:  $S_{alert} \leftarrow \{d \in M : h[d] > \tau\}$ 
6:  $E \leftarrow \{(d_i, d_j) : cosine(E(d_i), E(d_j)) \geq \epsilon\}$ 
7:  $C^* \leftarrow BronKerbosch(Graph(M, E))$ 
8: if  $|C^*| \geq k_{min}$  and  $provenance(C^*) \neq user$  then
9:    $S_{alert} \leftarrow S_{alert} \cup C^*$ 
10: end if
11: return  $S_{alert}$ 

```

D6 工具调用任务对齐检查用于防止候选记忆与用户目标不一致，尤其防止表面上是“长期偏好”或“操作说明”的内容在未来诱导 Agent 调用不相关工具。该模块把候选内容、用户目标与工具意图放在同一语义空间中比较，确保写入内容确实服务于用户任务。

算法 14: 类型二 D6 工具调用任务对齐检查

输入: 写入动作 a , 用户原始目标 Q_{user} , 对齐阈值 θ_{align}

输出: $v_6 \in \{\text{PASS}, \text{BLOCK}\}$

注: 该步骤防止候选记忆把普通用户任务升级为高风险工具动作。

```
1:  $align \leftarrow \text{cosine}(E(\text{verbalize}(a)), E(Q_{user}))$ 
2: if  $align < \theta_{align}$  then
3:   return Block
4: end if
5: return Pass
```

2.4.3 类型二端到端流程

算法 15 将 D1、D2、D3、D4 和 D6 串联为 Agent 自主写入路径。该流程体现了“先判断是否异常诱导，再判断是否因果依赖，再模拟未来后果，最后绑定来源并写入”的工程顺序。

算法 15: 类型二 AgentImmune 写入路径防御

输入: 写入上下文 $wc = \langle user_goal, indication, candidate, tq \rangle$

输出: $verdict \in \{\text{PASS}, \text{BLOCK}\}$

注: 该流程把意图检测、因果归因、仿真验证和任务对齐接入写入热路径。

```
1:  $d_1 \leftarrow indication$  if  $indication \neq \emptyset$  else  $tq$ 
2:  $v_1 \leftarrow D_1.\text{check}(d_1)$ 
3: if  $v_1 = \text{BLOCK}$  then
4:   return Block
5: end if
6:  $sig \leftarrow \text{HMAC}_k(candidate || \ell_r || \text{SHA256}(tq))$ 
7:  $tag \leftarrow (\ell_r, sig)$ 
8:  $v_2 \leftarrow D_2.\text{check}(wc)$ 
9: if  $v_2 = \text{BLOCK}$  then
10:  return Block
11: end if
12: if  $v_2 = \text{FLAG}$  then
```

```

13:    $v_3 \leftarrow D_3.\text{check}(\text{candidate})$ 
14:   if  $v_3 = \text{BLOCK}$  then
15:       return Block
16:   end if
17: end if
18:  $v_6 \leftarrow D_6.\text{check}(\text{candidate}, \text{user\_goal})$ 
19: if  $v_6 = \text{BLOCK}$  then
20:     return Block
21: end if
22:  $\text{memory.insert}(\text{candidate}, \text{tag})$ 
23: return Pass

```

在记忆进入长期库之后，类型二仍需要检索期监控。算法 16 对近期查询日志进行统计，发现过度命中的 Hub 条目与异常语义团簇，并对其执行验签和隔离。这样，AgentArmor 能够阻断风险内容从“已写入”进一步演化为“跨任务高频命中”。

算法 16: 类型二 AgentImmune 检索路径防御

输入： 记忆库 M ，近期查询日志 Q_{recent}

输出： 可疑记录集合 S_{alert}

注：该流程在检索期发现被过度命中的 Hub 条目和异常语义团簇。

```

1: for all  $d \in M$  do
2:    $h[d] \leftarrow |\{q \in Q_{recent} : d \in \text{top}_k(q)\}|$ 
3: end for
4:  $\mu \leftarrow \text{mean}(h); \sigma \leftarrow \text{std}(h); \tau \leftarrow \mu + \alpha\sigma$ 
5:  $S_{alert} \leftarrow \{d \in M : h[d] > \tau\}$ 
6:  $E \leftarrow \{(d_i, d_j) : \text{cosine}(E(d_i), E(d_j)) \geq \epsilon\}$ 
7:  $C^* \leftarrow \text{BronKerbosch}(\text{Graph}(M, E))$ 
8: if  $|C^*| \geq k_{min}$  and  $\text{provenance}(C^*) \neq \text{user}$  then
9:    $S_{alert} \leftarrow S_{alert} \cup C^*$ 
10: end if
11: for all  $d \in S_{alert}$  do
12:   if  $\text{HMAC}_k(d.\text{content} || d.\text{label} || d.\text{hash}) \neq d.\text{sig}$  then

```

```

13:     quarantine( $d$ )
14:   end if
15: end for
16: return  $S_{alert}$ 

```

2.5 类型三：反思/合成写入防御模块

类型三面向反思器、总结器或经验归纳模块。风险内容可能混入网页、搜索结果、OCR 文本、日志片段和工具输出等低可信来源中，使反思器在后台总结时把低可信内容压缩为候选事实，并进一步固化为长期记忆。该过程本质上是一种“信任漂白”：原本不能直接被信任的外部内容，经由系统自己的总结语言包装后，被误认为是稳定可信的内部记忆。

类型三模块保护“原始内容 T 、候选事实 C 、长期记忆 M 、检索记忆 $\hat{M}$ 、Agent 动作 a ”这一完整链路：

$$T \xrightarrow{\text{Reflector}} C \xrightarrow{\text{D1-D5+Provenance}} M \xrightarrow{\text{Retrieval Guard}} \hat{M} \xrightarrow{\text{Agent Use}} a. \quad (2.2)$$

其中，D1-D5 负责写入期防护，Provenance 负责来源绑定和审计，R1 负责检索期防护。

2.5.1 反思写入前的意图筛查

D1 反思意图筛查部署在反思器输出候选事实之前，用于识别原始材料或摘要中是否存在面向“反思/记忆/总结”模块的操控意图。该模块不只检查原始外部文本，还同时检查反思器生成的摘要和候选事实，避免风险内容在摘要过程中被重新包装后逃逸检测。

算法 17: 类型三 D1 反思意图筛查

输入： 原始轮次 raw_turns ，摘要文本 $summary$ ，候选事实集合 $facts$

输出： $verdict \in \{\text{PASS}, \text{FLAG}, \text{BLOCK}\}$

注：该步骤用于识别原始材料或反思摘要中面向记忆模块的操控意图。

1: $joined \leftarrow \text{concat}(raw_turns, summary, facts)$

2: $score \leftarrow 0$

```

3: for all  $rule \in RULES$  do
4:   if  $rule\_hit(joined, rule)$  then
5:      $score \leftarrow score + rule.weight$ 
6:   end if
7: end for
8: if  $reflection\_directive$  与  $workflow$  或  $authority$  共现 then
9:    $score \leftarrow score + graph\_bonus$ 
10: end if
11: if  $hard\_block$  or  $score \geq block\_threshold$  then
12:   return Block
13: end if
14: if  $score \geq flag\_threshold$  then
15:   return Flag
16: end if
17: return Pass

```

D1 的输出直接影响后续 D2-D5：若结果为 BLOCK，候选摘要不会进入记忆写入流程；若结果为 FLAG，则系统继续执行摘要接地性审计、一致性校验和策略门控；若结果为 PASS，则候选事实仍需携带来源标签写入，避免低可信事实在后续阶段丢失来源信息。

2.5.2 反思记忆的检索期复核

反思写入期审查不能保证历史摘要在未来任务中始终安全。因此，类型三还在检索期执行 R1 防御，对已召回条目进行签名验证、来源重排、低完整性降权和异常簇过滤。其目标是避免低可信摘要在后续查询中被当作高置信证据直接进入上下文。

算法 18: 类型三 R1 检索期防御

输入： 查询 $query$ ，记忆排序器 $memory_rank$ ，候选记忆签名与来源标签

输出： 重新排序后的安全记忆条目 $reranked_entries$

注：该步骤在历史记忆被重新召回时执行验签、来源重排和异常簇过滤。

```

1:  $entries \leftarrow \text{top}_k(\text{memory\_rank}(query))$   ▷ 先取得常规检索结果，再做安全重排

```

```

2: for all entry  $\in$  entries do
3:   if verify_signature(entry) = false then
4:     block(entry)
5:   else
6:     weight  $\leftarrow$   $0.60 \cdot \text{query\_score} + 0.25 \cdot \text{provenance\_score} + 0.15 \cdot$ 
       integrity_label
7:     if low_integrity(entry) then
8:       downweight(entry)
9:     end if
10:    if hubness_abnormal(entry) or coordinated_cluster(entry) then
11:      flag_and_downweight(entry)
12:    end if
13:  end if
14: end for
15: return reranked_entries

```

2.6 类型四：外部/开发者批量写入防御模块

类型四发生在用户正常查询之前。外部低可信源或被污染的外部源通过上传、同步、API 写入、爬虫抓取、开发者批处理导入等路径，把带有风险指令、检索诱饵、危险 workflow 或伪事实的文档批量写入知识库。与对话式污染相比，类型四具有预先持久化、供应链污染、检索放大和跨阶段传播四个特点。风险载荷会先经历文档上传、嵌入、分块、索引、检索和拼接，最后才影响 LLM 响应或工具调用。

类型四模块实现 EDW（External / Developer Write）纵深防御，覆盖上传、分块、索引和检索三个阶段。SP1 是嵌入异常检测，对上传文档的向量表示进行分布偏离、异常相似度和离群程度分析。SP2 是内容困惑度分析，通过字符级或词级困惑度识别混入文档中的异常指令片段和格式突变。SP3 是跨块连贯性验证，用于判断分块后的相邻内容是否存在语义断裂或跨块拼接污染。SP4 是触发区域检测，扫描文档中高度集中的触发词、工具名、角色标签和高风险动作区域。SP5 是鲁棒聚合检索，降低单个异常簇垄断上下文的概率。SP6 是后检索一致性验证，用于检查检索结果中的行动建议、工具调用、身份规则和记忆事实是否与用户当前任务一致。SP7 是语义依赖图分析，尝试建模跨块、跨文档之间的依赖关系。

2.6.1 外部文档进入知识库前的连续门控

类型四的伪代码不宜拆成多个孤立算法，因为 SP1–SP7 对应的是同一条文档导入链路的不同检查点。算法 19 因此采用统一流程表达：文档先经过上传期筛查；只有通过嵌入异常和困惑度扫描的文档才会进入分块阶段；分块后继续执行连贯性、触发区域和依赖图检查；最后通过鲁棒检索与后检索一致性验证控制其进入模型上下文的方式。

算法 19: 类型四 EDW 外部批量写入纵深防御流程

输入： 外部文档集合 $\mathcal{D}$ ，知识库 $\mathcal{M}$ ，隔离队列 $\mathcal{Q}$

输出： 更新后的知识库 $\mathcal{M}$ 与隔离队列 $\mathcal{Q}$

注：该流程覆盖外部文档从上传到检索的完整知识库导入链路。

```
1: for all  $d \in \mathcal{D}$  do
2:    $z \leftarrow \text{Embed}(d)$  ▷ 上传阶段先提取文档整体语义表示
3:    $v_1 \leftarrow \text{SP1.embedding\_anomaly}(z)$ 
4:    $v_2 \leftarrow \text{SP2.perplexity\_scan}(d)$ 
5:   if  $v_1 = \text{BLOCK}$  or  $v_2 = \text{BLOCK}$  then
6:      $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(d, \text{upload\_risk})\}$ 
7:     continue
8:   end if
9:    $\text{chunks} \leftarrow \text{Chunk}(d)$  ▷ 仅对通过上传期筛查的文档执行分块
10:   $v_3 \leftarrow \text{SP3.coherence\_check}(\text{chunks})$ 
11:   $v_4 \leftarrow \text{SP4.trigger\_region\_scan}(\text{chunks})$ 
12:   $v_7 \leftarrow \text{SP7.dependency\_graph}(\text{chunks})$ 
13:  if  $v_3, v_4, v_7$  中存在高风险结果 then
14:     $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(d, \text{index\_risk})\}$ 
15:  else
16:     $\mathcal{M} \leftarrow \mathcal{M} \cup \text{Index}(\text{chunks})$ 
17:  end if
18: end for
19:  $R \leftarrow \text{SP5.robust\_retrieve}(\mathcal{M})$ 
20:  $\hat{R} \leftarrow \text{SP6.post\_retrieval\_verify}(R)$ 
```

21: **return** ($\mathcal{M}, Q, \hat{R}$)

该流程将“上传、分块、索引、检索”四个工程阶段连接起来，使外部文档不再绕过对话层直接进入知识库。即使某个文档片段没有在上传期被阻断，后续的分块连贯性检查、触发区域扫描和后检索一致性验证仍能提供二次防护。

2.7 类型五：Multi-Agent 共享写入防御模块

类型五面向多 Agent 协同系统。多个 Agent 通常共享对话日志、crew memory、RAG memory store 或共享知识库，以复用彼此的中间结果。共享记忆提升了协作效率，也放大了污染风险：低权限 Agent 读取外部网页或工具输出后，可能把风险内容写入共享记忆；高权限 Agent 在后续正常任务中检索到该记忆，并据此调用邮件、数据库、代码仓库、部署系统或命令执行工具。污染影响由此从“单个 Agent 的一次回答”升级为“跨 Agent、跨任务、跨权限的持久化触发”。

类型五模块以 MASW (Multi-Agent Shared Write) 为核心场景，设置七个防御节点。D1 是输入来源标记，为来自用户、网页、搜索结果、工具返回、文件和模型总结的内容打上来源标签。D2 是候选事实抽取，将 Agent 生成的中间结果降级为候选事实，而不是直接写入共享可信记忆。D3 是混合风险筛查，结合规则检测和语义 rubric 识别显性注入、隐蔽外传表达和伪装业务规则。D4 是共享写入网关，是进入共享记忆的核心边界。D5 是检索审计，使共享记忆检索结果只能作为 evidence，而不能天然成为 instruction。D6 是执行仲裁，专门保护邮件外发、数据库写入、仓库提交、生产部署、密钥读取等高影响工具。D7 是撤销与影响分析，用于在发现污染记忆后追踪其派生记忆、检索命中和工具影响范围。

2.7.1 共享记忆污染链路建模

算法 20 不是系统的防御逻辑，而是对 MASW 风险传播路径的形式化描述。它展示低权限 Agent 如何从外部内容中生成摘要并写入共享记忆，以及高权限 Agent 如何在未来任务中检索该记忆并形成高影响动作。该建模为后续写入网关、检索审计和执行仲裁确定了防护位置。

算法 20: 类型五 MASW 污染链路复现流程

输入： 外部内容源 d ，注入载荷 p ，低权限 Agent A_1 ，高权限 Agent A_2 ，共享

记忆 $\mathcal{M}$, 目标查询 q_t

输出: 高风险动作 a^* 或风险未触发标记 $\perp$

注: 该流程用于描述共享记忆污染的风险传播路径, 为后续防御节点提供建模基础。

```
1:  $d' \leftarrow \text{Inject}(d, p)$ 
2:  $s \leftarrow A_1.\text{ReadAndSummarize}(d')$ 
3: if  $\text{VulnerableWritePolicy}(s) = \text{true}$  then
4:    $\mathcal{M} \leftarrow \mathcal{M} \cup \{s\}$ 
5: end if
6:  $\mathcal{R} \leftarrow A_2.\text{Retrieve}(\mathcal{M}, q_t)$ 
7:  $a^* \leftarrow A_2.\text{PlanAction}(q_t, \mathcal{R})$ 
8: if  $a^*.tool \in \{email.send, database.write, deploy.production, secret.read\}$ 
   then
9:   return  $a^*$ 
10: end if
11: return  $\perp$ 
```

2.7.2 共享写入网关与候选事实筛查

针对上述链路, AgentArmor 首先在共享写入入口设置 Provenance-Gated Shared Write。算法 21 要求所有外部输入先经过来源标记, 然后由写入 Agent 抽取候选事实。候选事实必须通过风险分数、来源可信性、敏感性和指令性检查, 才能从临时中间结果晋升为共享记忆。

算法 21: 类型五 Provenance-Gated Shared Write

输入: 外部输入 x , 写入 Agent a , 共享记忆 $\mathcal{M}$, 隔离队列 $\mathcal{Q}$

输出: 更新后的 $\mathcal{M}$ 与 $\mathcal{Q}$

注: 该流程是共享记忆写入边界, 只有来源可信、风险可控的候选事实才能晋升为记忆。

```
1:  $x' \leftarrow \text{Label}(x)$  ▷ 所有共享写入内容必须先绑定来源标签
2:  $\mathcal{C} \leftarrow \text{ExtractCandidateFacts}(a, x')$ 
3: for all  $c \in \mathcal{C}$  do
```

```

4:    $r \leftarrow \text{RiskScore}(c)$ 
5:   if  $r > \tau_3$  or  $\text{InstructionLike}(c)$  then
6:      $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(c, \text{risk\_filter})\}$ 
7:     break
8:   end if
9:    $v \leftarrow E(c) \wedge S(c) \wedge \neg K(c) \wedge r \leq \tau_4$ 
10:  if  $v = \text{true}$  then
11:     $m \leftarrow \text{PromoteToMemory}(c, \text{provenance}(c))$ 
12:     $\mathcal{M} \leftarrow \mathcal{M} \cup \{m\}$ 
13:  else
14:     $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(c, \text{provenance\_gate})\}$ 
15:  end if
16: end for
17: return  $(\mathcal{M}, \mathcal{Q})$ 

```

共享写入网关内部调用 Hybrid Risk Screening 计算候选事实风险分。算法 22 将规则命中与语义 rubric 评分结合起来：规则层负责快速识别角色伪装、工具调用指令、外传表达等显性模式；rubric 层负责识别更隐蔽的业务规则伪装和跨 Agent 权限转移意图。输出的风险标签也会进入审计日志，便于后续复盘。

算法 22: 类型五 Hybrid Risk Screening

输入： 候选事实 c ，规则集合 Ω ，rubric 维度集合 Φ

输出： 风险分数 $R(c)$ 与风险标签集合 $\mathcal{B}$

注：该流程综合规则与语义 rubric，为候选事实计算可解释风险分。

```

1:  $R(c) \leftarrow 0; \mathcal{B} \leftarrow \emptyset$  ▷ 初始化可解释风险分与风险标签集合
2: for all  $\omega \in \Omega$  do
3:   if  $\omega.\text{match}(c.\text{text}) = \text{true}$  then
4:      $R(c) \leftarrow R(c) + \omega.\text{weight}$ 
5:      $\mathcal{B} \leftarrow \mathcal{B} \cup \{\omega.\text{tag}\}$ 
6:   end if
7: end for
8: for all  $\phi \in \Phi$  do

```

```

9:    $s_\phi \leftarrow \text{RubricScore}(c, \phi)$ 
10:  if  $s_\phi > \phi.\text{threshold}$  then
11:     $R(c) \leftarrow R(c) + \phi.\text{weight}$ 
12:     $\mathcal{B} \leftarrow \mathcal{B} \cup \{\phi.\text{tag}\}$ 
13:  end if
14: end for
15: if  $\text{ExternalRecipient}(c) \wedge \text{ActionVerb}(c)$  then
16:    $R(c) \leftarrow R(c) + w_e$ 
17:    $\mathcal{B} \leftarrow \mathcal{B} \cup \{\text{exfiltration}\}$ 
18: end if
19: return  $(R(c), \mathcal{B})$ 

```

2.7.3 检索审计、执行仲裁与撤销机制

当共享记忆被未来任务命中时，D5 检索审计将其视为 evidence，而不是天然可信的 instruction。算法 23 对检索结果进行验签、来源评分、任务相关性评分和风险降权，只有经过审计后的记忆才能进入高权限 Agent 上下文。

算法 23: 类型五 Trust-Aware Retrieval Audit

输入： 目标查询 q ，共享记忆 $\mathcal{M}$ ，最小信任阈值 τ_T ，最大风险阈值 τ_R

输出： 可进入上下文的记忆集合 $\mathcal{R}$

注：该流程将共享记忆检索结果降级为 evidence，防止其直接成为工具指令。

```

1:  $\mathcal{R} \leftarrow \emptyset$  ▷ 默认不信任任何共享记忆，逐条审核后放行
2: for all  $m \in \mathcal{M}$  do
3:   if  $\text{ScopeMismatch}(m, q) = \text{true}$  then
4:     continue
5:   end if
6:   if  $T(m) < \tau_T$  or  $R(m) > \tau_R$  then
7:      $\text{AuditLog}(m, q, \text{filtered})$ 
8:     continue
9:   end if
10:   $\text{score} \leftarrow \text{sim}(m, q) + \alpha T(m) - \beta R(m)$ 

```

```

11:    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(m, score, evidence\_only)\}$ 
12: end for
13:  $\mathcal{R} \leftarrow \text{TopK}(\mathcal{R})$ 
14: return  $\mathcal{R}$ 

```

若检索记忆即将影响高影响工具调用，仅靠检索审计仍不够。D6 执行仲裁在工具调用前再次检查动作权限、记忆风险、人工确认需求和执行日志。算法 24 保证共享记忆最多提供证据支持，而不能单独决定邮件发送、数据库写入、生产部署或密钥读取等高风险动作。

算法 24: 类型五 Execution Mediation for High-Impact Tools

输入： 动作提案 a ，用户目标 g ，检索上下文 $\mathcal{R}$ ，工具策略 Π

输出： 执行决策 $d \in \{allow, deny, review\}$

注：该流程保护邮件、数据库、仓库、部署和密钥读取等高影响工具。

```

1: if  $\neg \text{GoalAligned}(a, g)$  then                                ▷ 动作必须与用户当前目标一致
2:   return deny
3: end if
4: if  $\neg \text{PermissionOK}(a, \Pi)$  then
5:   return deny
6: end if
7: if  $\text{ExternalSink}(a) \wedge \text{SensitiveContext}(\mathcal{R})$  then
8:   return review
9: end if
10: if  $a.tool \in \{email.send, database.write, repo.commit, deploy.production, secret.read\}$ 
    then
11:   return review
12: end if
13: return allow

```

最后，类型五需要具备污染发现后的恢复能力。D7 撤销与影响分析通过反向依赖图追踪污染记忆的派生条目、检索命中记录和已触发工具动作。算法 25 将受影响条目隔离、将相关动作写入审计队列，并更新共享记忆库的信任状态。

算法 25: 类型五 Revocation and Impact Analysis

输入: 溯源图 $\mathcal{G}$, 污染记忆节点 p , 共享记忆 $\mathcal{M}$

输出: 撤销后的共享记忆 $\mathcal{M}'$ 与影响集合 $\mathcal{I}$

注: 该流程在发现污染条目后沿溯源图追踪派生影响并撤销相关记忆。

```
1:  $\mathcal{I} \leftarrow \{p\}; W \leftarrow \{p\}$ 
2: while  $W \neq \emptyset$  do
3:    $u \leftarrow \text{pop}(W)$ 
4:   for all  $v \in \text{Children}_{\mathcal{G}}(u)$  do
5:     if  $v \notin \mathcal{I}$  then
6:        $\mathcal{I} \leftarrow \mathcal{I} \cup \{v\}; W \leftarrow W \cup \{v\}$ 
7:     end if
8:   end for
9: end while
10: for all  $m \in \mathcal{I}$  do
11:    $\mathcal{M} \leftarrow \mathcal{M} \setminus \{m\}$ 
12:    $\text{AuditLog}(m, \text{revoked})$ 
13: end for
14: return  $(\mathcal{M}, \mathcal{I})$ 
```

第三章 作品测试与分析

本章节针对 AgentArmor 的系统实现开展功能验证与安全效果分析，重点评估其在长期记忆写入、记忆检索、多 Agent 共享传播和高危工具调用中的稳定性、完整性与防护有效性。测试过程面向真实工程链路展开，使用项目内实验脚本、原始数据表和审计日志生成结论；凡未能形成完整真实执行链路的项目，正文只说明边界条件，不以替代数值参与统计。

本轮测试覆盖端到端记忆污染链路、类型一至类型三历史模块复现实验、类型四知识库/供应链污染困难集、类型五共享记忆传播困难集、关键组件消融、良性语料误伤和本地运行开销。实验结果表明，AgentArmor 在本地 SQLite 记忆后端和安全工具沙箱中能够阻断由污染记忆引发的高危工具执行，但在语义低显著投毒、正常安全术语误伤和外部框架完整 Agent 链路接入方面仍存在需要后续扩展的边界。

本作品代码已开源至 GitHub: <https://github.com/panxiaogong/AgentArmor>。报告中的量化数据均可回溯到工程目录下的 CSV、JSONL、SQLite 或日志输出，便于复核实验过程、对照关键样本并复现实验结论。

整体来看，本章测试体现了 AgentArmor 的三项工程优势：其一，防护对象覆盖从候选记忆写入到未来检索、计划生成和工具执行的完整生命周期；其二，测试链路同时包含攻击样本、良性协作样本和误伤分析，能够衡量安全性与可用性的平衡；其三，系统在 SQLite、本地安全沙箱、常见索引框架和向量数据库适配实验中保持统一审计口径，说明该方案具备向真实 Agent 工程迁移的基础。

3.1 测试目标

本章围绕 AgentArmor 的记忆污染防护能力开展系统测试，重点验证长期记忆在写入、检索、共享传播和工具执行阶段的安全边界。测试设计遵循闭环验证原则：先确认无防护条件下攻击链能够真实触发，再比较 AgentArmor 的阻断效果、误报风险和运行开销。

表 3.1 测试目标与评价指标

测试维度	验证内容	评价指标
攻击链有效性	验证污染输入是否会进入长期记忆，并在后续检索和工具调用阶段持续产生影响。	恶意写入率、污染检索率、危险执行率
防护有效性	验证 AgentArmor 在写入、检索和执行三个关键边界的阻断能力。	写入阻断率、危险执行阻断率、撤销有效性
准确性与可用性	比较不同基线与消融配置在困难集上的检测能力，同时观察良性任务是否受影响。	Precision、Recall、F1、FPR、良性成功率
运行开销	在统一口径下统计本地实验耗时与资源使用，区分本地规则、SQLite 路径和远程模型调用。	P50、P95、吞吐量、模型调用数

3.2 实验环境与数据设置

实验采用统一脚本生成数据、执行评测并导出结果。端到端链路使用 SQLite 作为持久化记忆后端，工具调用均限制在本地安全沙箱内，不会外发邮件、部署生产环境、读取真实密钥或执行破坏性命令。原始样本、结果表和审计日志随工程目录保留，正文仅列示关键指标。

表 3.2 实验环境与数据规模

项目	设置	说明
运行环境	本地 Python；随机种子 20260703	所有本地结果均可由实验脚本重新生成。
记忆后端	SQLite 持久化记忆库	用于端到端写入、检索、审计与撤销实验。
工具执行	本地安全沙箱	覆盖文件写入、SQLite 查询、邮件草稿、模拟仓库提交、模拟部署与模拟密钥访问。
端到端样本	30 条恶意链路，8 条良性链路	验证污染记忆对后续任务和工具调用的持续影响。
类型四数据	主集与困难集	覆盖文档污染、注释投毒、语义改写、中英文混合和来源冲突。
类型五数据	72 条恶意样本，64 条良性协作样本	覆盖多 Agent 共享记忆、权限差异、横向传播和撤销后检索。
良性语料	216 条记忆样本	用于估计正常安全讨论、工具文档和协作消息中的误伤风险。

本章分析按照“链路验证、场景评测、组件消融、误伤定位、运行开销”五个层次展开。端到端实验用于证明污染记忆能够在无防护条件下跨轮传播，并检验 AgentArmor 是否能在写入网关、检索审计和执行仲裁处形成闭环；类型四和类型五困难集用于观察系统面对知识库污染、多 Agent 共享记忆和工具调用风险时的综合表现；消融实验进一步拆解 D3/D4/D6 等关键组件对检测率、误伤率和危险执行控制的贡献。

表 3.3 类型一至类型三复现实验结果

场景	模块与数据	主要指标	实验结论
类型一	AutoWrite; 120 条自写入样本, Config-5 Full	Precision=1.000; Recall=0.312; F1=0.476; FPR=0.000	完成。42 项单元测试通过; 结果来自本地确定性评估, 未调用远程 LLM。
类型二	MINJA; 120 条自主写入与检索样本, Config-5	Precision=1.000; Recall=1.000; F1=1.000; FPR=0.000	完成。52 项单元测试通过; 覆盖写入、检索与后续任务读取路径。
类型三	Reflection; 反思写入与检索样本, Config-6	Precision=0.815; Recall=1.000; F1=0.898; FPR=0.250	完成。15 项单元测试通过; 严格检索策略提高召回, 但对正常反思内容较保守。

类型一实验用于验证 Agent 在自动写入记忆时是否会将攻击指令固化为可复用事实。该模块的 Precision 为 1.000, 说明被判定为攻击的样本较为集中; Recall 为 0.312, 反映出仅依靠写入前规则仍难覆盖低显著表达。类型二实验覆盖自主写入与后续检索路径, F1 达到 1.000, 说明在该协议下写入和检索边界均能稳定复现。类型三实验面向反思型记忆, FPR 为 0.250, 表明严格策略会把部分正常反思内容纳入风险集合, 因此该结果主要用于说明防护上限和误伤边界。

表 3.4 外部框架与向量数据库接入验证结果

实验项	状态	验证内容	结果处理
LangChain Core	完成	本地向量存储写入 2 条，检索命中攻击样本；写入 0.032 ms，检索 0.335 ms	作为本地集成验证记录，不并入类型四/类型五统一指标。
LlamaIndex Core	完成	本地索引写入 2 条，检索命中攻击样本；写入 53.324 ms，检索 0.574 ms	作为本地集成验证记录，不并入类型四/类型五统一指标。
AutoGen AgentChat	部分完成	runtime 与消息对象构造成功；构造 0.103 ms	仅证明本地 runtime 对象可构造，不作为完整 Agent 链结果。
ChromaDB	完成	持久化集合写入 3 条，检索命中攻击样本；写入 7.268 ms，检索 2.015 ms	作为本地集成验证记录，不并入类型四/类型五统一指标。
FAISS	完成	索引写入 3 个向量，最近邻返回攻击向量；写入 0.371 ms，检索 1.658 ms	作为本地集成验证记录，不并入类型四/类型五统一指标。
Qdrant Client	完成	内存集合写入 2 个点，检索命中攻击点；写入 1.552 ms，检索 0.47 ms	作为本地集成验证记录，不并入类型四/类型五统一指标。

外部接入验证用于说明 AgentArmor 与常见索引框架和向量数据库之间的工程适配状态。LangChain Core、LlamaIndex Core、ChromaDB、FAISS 和 Qdrant 均已完成本地写入与检索闭环，可证明记忆候选进入外部存储后仍能被检索命中；AutoGen 已完成 runtime 与消息对象构造验证。该组实验说明，AgentArmor 的核心防护逻辑可以围绕“候选记忆进入外部存储前”和“外部存储召回记忆后”两个关键位置部署，具备继续接入真实 Agent 任务链的工程基础。

3.3 端到端攻击链验证

端到端实验用于检验污染记忆是否会跨轮影响 Agent 行为。无防护模式允许候选记忆直接写入并参与后续检索；AgentArmor 模式在写入网关、检索审计和高影响工具执行前进行联合判定。

表 3.5 端到端攻击链验证结果（样本计数）

模式	恶意写入	污染检索	危险执行	写入阻断	良性成功	P95/ms
无防护	30/30	30/30	27/30	0/30	8/8	0.697
AgentArmor	0/30	0/30	0/30	30/30	8/8	0.843

图中展示污染记忆从写入到高危工具调用的完整链路及三处防护边界。

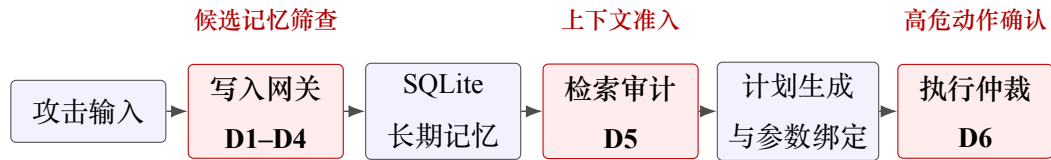


图 3.1 端到端记忆污染链路及 AgentArmor 防护边界

实验结果表明，无防护模式下 30 条恶意链路全部完成恶意写入和污染检索，其中 27/30 进一步触发危险工具动作。启用 AgentArmor 后，恶意写入、污染检索和危险执行均为 0/30，8 条良性任务均完成预期流程。该结果说明，在本轮构造链路中，写入网关阻止污染进入长期记忆，执行仲裁又为后续计划偏移提供最后一道确认边界；因此危险执行为 0/30 是样本级观测值，而不是对任意线上输入的绝对化断言。

从工程含义看，端到端链路验证的价值不只在于最终危险工具是否执行，还在于复现了“写入-检索-计划-工具”的连续传播过程。无防护路径中，污染内容先被当作候选记忆保存，再在后续任务中被检索并影响计划参数；受保护路径中，候选记忆在写入前被判定为高风险，后续检索结果不再携带攻击载荷，即使计划阶段出现高影响动作，也需要通过 D6 仲裁。这说明 AgentArmor 的防护不是一次性输入过滤，而是覆盖长期状态、上下文准入和高影响动作确认的多阶段记忆安全机制。

3.4 场景化评测结果

统一量化结论以同一脚本、同一数据协议下的端到端链路、类型四困难集和类型五困难集为核心；类型一至类型三属于独立协议下的复现实验，其样本来源、配置命

名和攻击协议与类型四、类型五不同，因此单独呈现，不混入统一排名。

表 3.6 主要测试结果汇总

验证对象	数据规模	关键结果	结论
端到端攻击链	30 条恶意链路、8 条良性链路	危险执行 0/30，良性成功 8/8	写入网关与执行仲裁能够阻断本轮完整污染链路。
类型一至三复现实验	3 个历史模块本地复现	Type1 F1=0.476；Type2 F1=1.000；Type3 F1=0.898	结果来自真实脚本输出；因评测协议不同，单列展示，不与类型四、类型五混合排名。
类型四困难集	118 条评测样本	检出 36/54，误伤 8/64，F1=0.735	优于关键词和困惑度基线，但仍存在语义低显著攻击漏检。
类型五困难集	72 条恶意样本、64 条良性样本	检出 72/72，危险执行 0/72，F1=0.900	完整配置可阻断本轮共享记忆到高危工具的传播。

3.4.1 类型四：供应链与知识库污染

类型四实验面向外部文档、知识库片段和代码注释中的间接投毒。该场景的难点在于恶意内容往往不直接出现命令式攻击语句，而是通过摘要、注释或来源冲突影响后续检索。

图中展示外部知识片段进入长期记忆前后的核查流程。

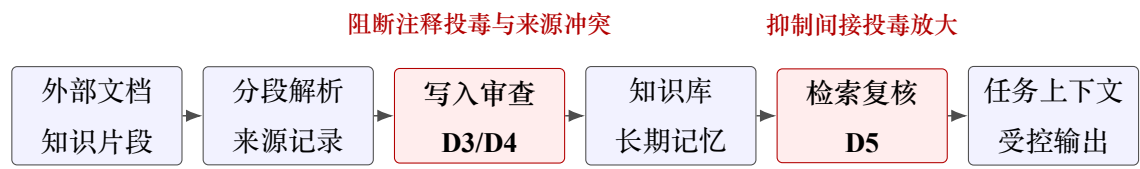


图 3.2 类型四知识库污染防护流程

表 3.7 类型四困难集检测效果

方法	状态	Precision	Recall	F1	FPR	P95/ms
关键词规则	完成	0.000	0.000	0.000	0.000	0.005
困惑度基线	完成	1.000	0.111	0.200	0.000	0.008
AgentArmor 完整配置	完成	0.818	0.667	0.735	0.125	0.007
去隐藏区检测	完成	0.818	0.667	0.735	0.125	0.006
去检索期核查	完成	0.818	0.667	0.735	0.125	0.006

在困难集上，AgentArmor 完整配置取得 $F1=0.735$ ，检出 36/54 条恶意样本，误伤 8/64 条良性样本，高于关键词规则和困惑度基线。关键词规则对隐蔽语义改写的覆盖不足，困惑度基线能够捕捉部分异常表达，但对注释投毒、摘要间接投毒和中英文混合样本仍不稳定。AgentArmor 的优势在于将来源、语义风险和检索期上下文一起纳入判断，使知识片段不再只按字面关键词进入长期记忆。

类型四结果同时暴露出两个限制。第一，低显著性投毒会把攻击意图伪装成“格式修正”“摘要补充”或“兼容性说明”，单靠静态规则难以充分覆盖。第二，正常安全讨论、工具文档和代码注释中也会频繁出现 secret、token、query、email 等词，保守策略会带来误伤。因此， $F1=0.735$ 不是满分结果，而是说明当前组合策略已优于简单基线；后续仍需要结合来源可信度、文档签名、任务上下文和人工复核降低漏报与误报。

3.4.2 类型五：多 Agent 共享记忆

类型五实验检验共享记忆在多 Agent 协作中的横向传播风险。攻击样本覆盖跨 Agent 写入与检索、权限差异、工具说明伪装、危险参数注入和撤销后检索等情况。

图中展示多 Agent 共享记忆中污染传播的进入、扩散与撤销路径。

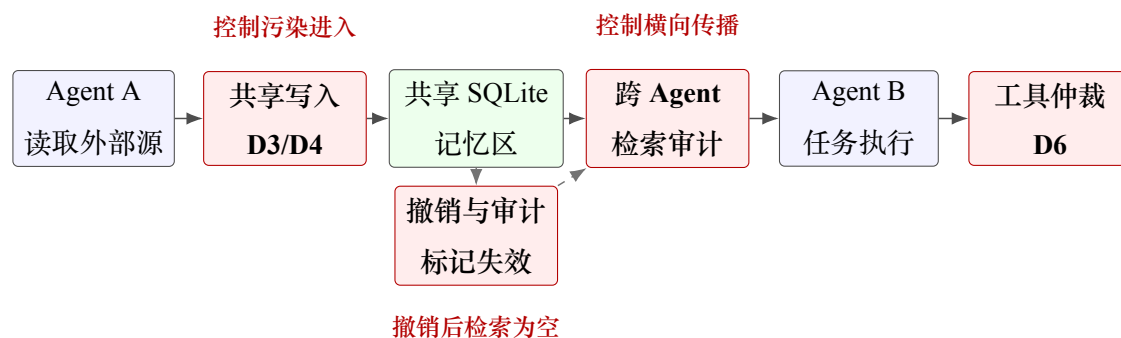


图 3.3 类型五共享记忆传播与阻断路径

表 3.8 类型五共享记忆防护效果（样本计数）

配置	检出 (TP/72)	误伤 (FP/64)	F1	危险执行	良性成功
无防护	0/72	0/64	0.000	72/72	64/64
仅 D3	30/72	16/64	0.508	72/72	64/64
仅 D4	48/72	16/64	0.706	72/72	64/64
D3+D4	48/72	16/64	0.706	72/72	64/64
完整配置	72/72	16/64	0.900	0/72	64/64

完整配置在类型五困难集上取得 F1=0.900、Recall=1.000，检出 72/72 条恶意样本，并将危险执行控制为 0/72。从表中可以看到，无防护配置虽然不产生检测告警，但 72/72 条恶意共享记忆都会继续触发高危执行；仅启用 D3 或 D4 时，系统已经能够识别部分污染进入点，但危险执行仍未被切断；完整配置加入检索审计和 D6 执行仲裁后，传播链路在工具边界前被收束，体现了多阶段防护闭环的必要性。

类型五结果还展示了 AgentArmor 对“安全性-可用性”平衡的处理方式。完整配置保持 64/64 条良性协作流程成功，同时记录 16/64 条良性样本进入风险集合，主要集中在正常安全描述、工具文档和协作消息中包含敏感动作词的情况。撤销实验共 20 条，其中 20 条在撤销后检索结果为空，说明系统不仅能在传播前阻断风险，也能在污染发现后通过撤销与审计机制收敛影响范围。

3.5 消融与误伤分析

消融实验用于识别关键防护环节。所有配置在相同类型五困难集上重复运行三组随机种子，表中列出核心配置的均值。

表 3.9 类型五关键组件消融结果

配置	F1 均值	Recall 均值	FPR 均值	95% CI(F1)
完整配置	0.900	1.000	0.250	0.000
去 D4 写入网关	0.692	1.000	1.000	0.000
去 D6 执行仲裁	0.706	0.667	0.250	0.000
仅 D3+D4	0.706	0.667	0.250	0.000
无防护	0.000	0.000	0.000	0.000

完整配置的 F1 均值为 0.900。去除 D4 写入网关后 FPR 上升至 1.000，说明共享写入边界是误报控制与污染隔离的关键环节；去除 D6 执行仲裁后 Recall 降至 0.667，说明仅依靠写入期检测不足以覆盖后续工具调用风险。

表 3.10 良性语料误伤来源

来源类型	误伤数	占良性语料比例	主要原因
普通代码/命令	12	5.6%	示例命令中含敏感词或模拟执行描述
安全讨论	12	5.6%	包含 token、password、secret 等安全术语
会议或普通记忆	6	2.8%	会议摘要中出现邮件草稿等协作动作
工作流记忆	6	2.8%	正常流程中包含 sqlite、query 等工具术语
工具文档	6	2.8%	工具说明提及 email、send 等动作词

误伤主要来自安全讨论、工具文档和普通代码示例中的敏感词。该结果说明，实际部署时需要结合来源可信度、任务上下文和人工复核机制，避免将正常安全说明误判为攻击指令。

3.6 运行开销

性能测试采用统一口径记录本地执行耗时。外部向量库接入验证只用于证明依赖接入和本地写入/检索路径可运行，不与类型四、类型五主实验的检测指标混合计算；未形成完整远程判别闭环的项目不列入延迟统计。

表 3.11 运行开销统计

方法	类型	网络	模型	延迟/ms	口径说明
端到端无防护	本地	否	否	P95=0.697	本地测量，不含模型推理
端到端 AgentArmor	本地	否	否	P95=0.843	本地测量，不含模型推理
AgentArmor 完整配置	本地	否	否	P95=0.008	本地测量，不含模型推理
类型五完整配置	本地	否	否	P50=0.615	单 Agent 本地中位耗时

在本地环境下，AgentArmor 端到端 P95 延迟为 0.843 ms，与无防护模式处于同一量级。需要注意的是，该结果不包含远程模型推理和线上向量数据库网络延迟；接入生产级外部系统后仍必须重新评测。

3.7 综合结论与扩展方向

综合以上实验，AgentArmor 在本地 SQLite 记忆后端和安全工具沙箱中能够有效阻断长期记忆污染导致的后续工具执行风险。端到端实验中，受保护路径没有恶意样本越过执行边界；类型五完整配置在 72 条恶意共享记忆样本中未触发高危工具动作，同时保持 64/64 条良性协作流程成功；类型四完整配置相较关键词和困惑度基线具有更好的综合检测效果。由此可见，AgentArmor 的优势不仅体现在单点检测准确率上，更体现在将写入筛查、检索复核、执行仲裁、撤销审计和误伤分析组织成可复现的安全闭环。

从作品价值看，AgentArmor 将长期记忆安全从“文本过滤”推进到“状态治理”：写入阶段控制污染进入，检索阶段控制上下文准入，执行阶段控制高影响工具动作，撤销阶段控制污染后的影响收敛。测试结果显示，该设计能够覆盖多轮对话、共享记忆和工具调用等 Agent 系统中最容易被忽视的风险链路，为后续接入更多框架、扩展更大规模数据集和部署真实业务 Agent 提供了清晰的工程基础。

表 3.12 后续扩展方向

方向	当前基础	扩展价值
统一评测协议	已完成类型一至类型三历史模块复现，并形成类型四、类型五困难集评测口径。	建立更完整的 benchmark 后，可支持跨场景横向比较和持续回归测试。
外部生态接入	LangChain Core、LlamaIndex Core、ChromaDB、FAISS、Qdrant 已完成本地最小写入/检索验证。	接入真实 Agent 任务、工具链和外部服务后，可进一步验证 AgentArmor 的框架无关性。
数据规模扩展	当前困难集覆盖文档污染、共享记忆传播、撤销后检索和良性协作语料。	引入真实业务数据和第三方公开数据集后，可进一步提升结论覆盖面。
误伤治理	已定位安全术语、工具说明和协作消息中的主要误伤来源。	结合来源白名单、任务上下文和人工复核策略，可进一步提升可用性。

第四章 创新性说明

4.1 问题定义层面的创新：从“输入安全”到“记忆安全”的范式转变

现有大模型安全研究主要集中于单次交互中的 Prompt Injection 检测、越狱识别与内容过滤，其防护对象本质上是“当前输入是否安全”这一瞬态问题。AgentArmor 提出了一个全新的安全问题视角：当 AI Agent 具备持久化长期记忆后，被污染的记忆本身就成为一种持续存在的安全威胁载体。

这一问题定义具有本质上的创新性：传统安全防护的污染影响随对话结束而消失，而记忆污染风险的效果可跨越多次会话、持续潜伏；风险输入发起方不需要在当前对话中直接破坏 Agent 行为，只需将恶意内容“种植”入记忆库，等待未来被检索时再触发；即使用户当前输入完全无害，Agent 仍可能因读取到被污染的记忆而产生危险输出，传统单点防护对此完全失效。

AgentArmor 是目前少有的将“记忆持久化污染”作为独立安全威胁模型进行系统性建模和防御的工程框架，填补了 Agent 安全领域从“单次防护”到“状态防护”的研究空白。

4.2 架构层面的创新：独立记忆安全网关（Memory Firewall）

AgentArmor 以 API Gateway 的形式构建了一个独立的 Memory Firewall 层，而非将安全逻辑内嵌进特定 Agent 框架或大模型实现中。这一架构设计具有以下创新价值：

解耦性：AgentArmor 与具体 LLM（如 GPT-4、Claude）、Agent 框架（如 LangChain、LlamaIndex）及向量数据库（如 ChromaDB、Pinecone）均无强绑定，可作为通用中间层接入任意 Agent 系统。

统一管控点：所有记忆的写入与读取请求必须经过 AgentArmor Gateway，实现了对 Agent 长期记忆的集中治理，避免因 Agent 内部逻辑绕过导致安全策略失效。

最小侵入性：现有 Agent 系统只需将记忆操作从直接访问向量数据库改为调用 AgentArmor API，即可获得完整的安全防护能力，改造成本极低。

横向扩展性：Gateway 本身基于 FastAPI 构建，天然支持异步处理与水平扩展，适应企业级大规模 Agent 部署场景。

这种“安全网关”的设计思路借鉴了网络安全领域防火墙的核心理念，并首次将其

系统性地应用于 AI Agent 记忆层的安全治理，是一种具有工程落地价值的架构创新。

4.3 检测机制的创新：同步规则过滤与异步语义免疫的分层协同防御

AgentArmor 创新性地将记忆安全检测分为两条并行路径，形成分层协同防御体系。

4.3.1 同步规则过滤层

写入热路径上的同步过滤器负责以极低延迟拦截可被模式匹配识别的明确风险，覆盖以下风险类型：指令覆盖型 Prompt Injection（如“Ignore all previous instructions”）、DAN / Jailbreak 越狱模式、Developer Mode 类角色伪装、Markdown 角色头注入（如伪造的 [SYSTEM]、[ASSISTANT] 标签）、中英文混合的伪造系统提示词。该层设计原则是“快速、确定、低延迟”，对已知风险模式实现毫秒级阻断，不引入额外 LLM 调用开销。

4.3.2 异步语义免疫层

对于可能绕过规则匹配的复杂语义风险，AgentArmor 引入了基于双记忆库（Attack Memory Bank 与 Benign Memory Bank）的向量语义免疫检测机制。这一机制将新写入的记忆嵌入为向量，分别计算其与恶意样本库、良性样本库的余弦语义距离，并根据距离差异将样本三分类为 attack（风险）、benign（良性）、candidate（待定）；对无法明确判断的 candidate 样本，则触发更深层的主动免疫流程。

两层检测路径的分工使系统在保证写入性能的同时，不丧失对隐蔽语义风险的识别能力，是安全性与可用性之间的工程平衡创新。

4.4 主动免疫机制的创新：基于 LLM 的沙箱模拟推理验证

AgentArmor 的主动免疫（Active Immunity）机制是本项目最具原创性的技术创新之一。对于语义距离无法明确判定安全性的 candidate 样本，系统引入了双角色 LLM 推理验证流程：

Simulation Agent：在隔离的沙箱语境中，将该候选记忆注入对话上下文，模拟 Agent 实际被检索该记忆后可能产生的回答。

Reflection Agent：独立扮演安全审查员角色，依据安全策略对 Simulation Agent 的输出进行风险评估，判断该回答是否存在信息泄露、指令劫持或其他危险行为。

这一机制的创新意义体现在以下三点：第一，将风险效果预演前置，不依赖对风险内容的静态分析，而是直接模拟“如果该记忆被真实使用，会产生什么危害”，从风险效果而非风险形式角度进行判定；第二，有效应对语义混淆风险，对于那些表面无害但在特定上下文中会引发危险行为的隐式注入，主动免疫机制能够通过动态推理发现其危害性，突破静态规则和语义距离的识别上限；第三，双角色对抗验证引入了相互独立的检验视角，降低了单一模型在特定场景下误判的概率。

这是将 LLM-as-Judge 思想与 Agent 记忆安全领域的首次深度结合，开辟了记忆安全检测的新技术路径。

4.5 数据结构的创新：可信记忆对象（Trusted MemoryEntry）

传统向量数据库通常直接存储原始文本及其嵌入向量。AgentArmor 将记忆条目重新定义为一个结构化可信对象（MemoryEntry），这是数据层面的根本性创新。

每条 MemoryEntry 不只包含记忆内容，还内置了完整的信任基础设施：content_hash 字段存储内容哈希，用于检测内容是否被篡改；source_id 与 source_type 字段精确标记记忆来源，区分用户输入、工具输出与系统生成；trust_score 字段提供量化信任分，支持基于来源动态调整可信程度；cryptographic_sig 字段存储 Ed25519 签名，覆盖关键 provenance 字段；is_unsafe 字段为安全状态标记，支持隔离而非删除的安全处置策略；quarantine_reason 字段提供结构化隔离原因，便于审计复盘；audit_trail 字段维护追加式审计链，记录完整生命周期事件。

这种设计使记忆条目从“数据库里的一段文本”升级为“一个具有来源可溯、内容可验、状态可查、生命周期可审计的可信对象”，为 AI Agent 记忆系统引入了类似可信账本的数据范式。

4.6 密码学应用的创新：Ed25519 来源签名保障记忆完整性

AgentArmor 将密码学完整性验证引入 AI 记忆系统，是安全工程领域跨界应用的创新实践。系统使用 Ed25519 非对称签名算法对每条记忆的关键 provenance 字段（包括 entry_id、content_hash、source_id、source_type、session_hash、timestamp、trust_score）进行签名。

其创新价值体现在以下几个方面：第一，防止事后篡改，若风险输入发起方在记忆写入后修改内容、伪造来源或调整信任分数，签名校验将立即失败，提供可检测的

完整性保证；第二，来源不可伪造，签名与 `source_id` 绑定，确保记忆的归因不可被风险输入发起方伪造或替换；第三，性能友好，Ed25519 是目前已知签名性能最优的椭圆曲线算法之一，单次签名操作在微秒级别完成，不影响写入路径的响应延迟；第四，密钥自动管理，系统在首次启动时若无配置密钥，会自动生成密钥对并引导写入环境变量，降低密码学工程门槛。

将传统 PKI 完整性保护机制应用于 AI Agent 记忆层的安全建模，是密码学工程与 AI 安全领域交叉创新的有益探索。

4.7 审计机制的创新：哈希链接的追加式审计链

AgentArmor 为每条记忆维护的 `append-only audit trail`，采用了类似区块链的哈希链接设计，是记忆可审计性方面的技术创新。每个审计事件包含前序事件的哈希值，形成不可篡改的事件因果链：从 `CREATED`（创建）到 `FILTER_PASSED / FILTER_BLOCKED`（过滤结果），再到 `IMMUNE_CHECK_PASSED / IMMUNE_CHECK_FLAGGED`（免疫检测结果），直至 `QUARANTINED`（隔离）与 `TRUST_SCORE_UPDATED`（信任分更新）。

这一机制的创新点在于：防止审计日志被选择性删除——如果风险输入发起方或内部人员试图删除某条隔离事件，哈希链断裂可立即被检测；事件时序不可重排——审计链通过哈希依赖关系固定事件顺序，防止日志被重新排列以掩盖操作序列；全生命周期覆盖——完整记录记忆从诞生到隔离的每一个关键操作节点；支持事后取证——结合结构化 JSONL 审计日志，可精确还原任意记忆条目的完整操作历史，为攻防复盘、安全审查和法律取证提供可信证据链。

4.8 防护时效的创新：周期扫描实现持续安全而非一次性过滤

现有安全防护通常是“写入时过滤，写入后不管”的一次性模型。AgentArmor 引入了后台周期扫描机制，将安全防护从“单点检测”升级为“持续巡检”。

这一创新应对了传统防护无法处理的以下场景：延迟激活污染——恶意内容在初次写入时绕过了当时的检测规则，但随后安全规则更新后，周期扫描可对其重新检测并补救隔离；渐进式语义污染——恶意来源可能通过多次写入表面无害的内容，组合形成危险语义，周期扫描可发现单条无法识别但组合危险的记忆模式；模型能力升级后的回溯审查——当免疫检测模型升级后，可对历史记忆重新执行更高精度的安全评估；记忆库长期健康维护——对大规模长期运行的 Agent 系统，周期扫描提供了

类似“定期体检”的健康保障机制。

这将 AgentArmor 从一个“边界防护过滤器”升级为一个具有持续免疫能力的动态安全系统，是面向长期运行 Agent 的安全工程创新。

4.9 隐私保护策略的创新：语义保留的 PII 动态脱敏

AgentArmor 的 PII 处理策略不是简单丢弃含隐私信息的记忆，而是在保留语义价值的前提下执行最小化脱敏替换，这是隐私保护与记忆可用性之间的工程创新。系统支持识别邮箱、中国大陆手机号、身份证号、信用卡号、IPv4 地址等多类 PII；使用语义占位符（如 [EMAIL_REDACTED]、[PHONE_CN_REDACTED]）替换原始值，保留记忆的语境结构；脱敏在写入路径中同步执行，保证隐私数据永不持久化至向量数据库；脱敏事件作为警告信息附加于写入响应，使 Agent 可感知脱敏发生并按需处理。

相较于直接拒绝含 PII 内容的简单策略，这一方案在合规性与记忆系统实用性之间实现了更精细的平衡。

4.10 整体创新性总结

AgentArmor 的创新性可从三个维度归纳：

问题维度：首次将“AI Agent 长期记忆污染”作为独立安全威胁模型进行系统性定义和防御，填补了 Agent 安全从单次交互防护到跨会话状态防护的研究空白。

技术维度：创新性地融合了同步规则过滤、语义向量免疫、LLM 沙箱主动验证、Ed25519 密码学签名、哈希链审计追踪和后台周期扫描六种技术手段，构建了多层纵深防御体系，各层互补、协同增效。

工程维度：以框架无关的 API Gateway 形式实现，最小侵入性接入现有 Agent 系统，兼顾了安全能力的完整性与工程落地的可行性，为 AI Agent 安全基础设施的产品化提供了可参考的架构范式。

第五章 总结

参考文献

- [1] 李建华. 网络空间威胁情报感知、共享与分析技术综述 [J]. 网络与信息安全学报, 2016, 2(2): 16–29. (样例, 参考国标 GB/T 7714–2015)